Università degli Studi di Padova

Dipartimento di Matematica «Tullio Levi-Civita»

Corso di Laurea in Informatica



Interfaccia UI per dispositivi ESP32 per la visualizzazione di informazioni in tempo reale

Tesi di Laurea

Relatore
Prof. Zanella Marco

Laureando
Roberto Mariano Doroftei
Matricola 2111031

Anno Accademico 2025-2026

© Roberto Mariano Doroftei, Ottobre 2026. Tutti i diritti riservati. Tesi di Laurea: «Interfaccia UI per dispositivi ESP32 per la visualizzazione di informazioni in tempo reale», Università degli Studi di Padova, Dipartimento di Matematica «Tullio Levi-Civita», Corso di Laurea in Informatica.

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di trecento ore, dal laureando Roberto Mariano Doroftei (matricola 2111031) presso l'azienda Spazio Dev S.r.l.. Lo stage è stato condotto sotto la supervisione del tutor aziendale Matteo Forzan, mentre il Prof. Zanella Marco ha ricoperto il ruolo di tutor accademico.

Questa tesi tratta la progettazione e lo sviluppo di un'interfaccia utente su microcontrollori ESP32 dotati di display, finalizzata alla visualizzazione in tempo reale delle informazioni gestite da RelAi, il CRM aziendale di Spazio Dev S.r.l.. Il sistema permette di esporre dati chiave quali codici QR, prenotazioni di eventi, il contatore degli iscritti a un'organizzazione e il contatore dei partecipanti a un evento direttamente presso le sedi delle organizzazioni clienti.

Lo scopo del progetto è duplice: da un lato, realizzare un firmware robusto e ottimizzato per dispositivi *embedded* con rigidi vincoli di memoria e risorse grafiche; dall'altro, implementare una connettività affidabile tramite provisioning Wi-Fi, gestione della cache locale dei dati e sincronizzazione periodica con le API aziendali, garantendo la continuità operativa anche in condizioni di assenza di rete. Il testo è suddiviso in otto capitoli. Il primo capitolo introduce il contesto aziendale, gli obiettivi del progetto e la pianificazione complessiva dello stage. Il secondo capitolo descrive i processi, le metodologie di sviluppo adottate e le modalità di interazione e revisione con il tutor aziendale. Il terzo capitolo presenta l'analisi degli utenti, i casi d'uso e i requisiti del sistema. Il quarto capitolo illustra le tecnologie hardware e software utilizzate per lo sviluppo del prototipo. Il quinto capitolo descrive l'architettura del firmware e l'implementazione dei moduli di connettività e provisioning. Il sesto capitolo tratta le attività di verifica e validazione del sistema. Il settimo capitolo riassume i risultati conseguiti e le conclusioni, mentre l'ottavo capitolo raccoglie la bibliografia e i riferimenti sitografici.

Ringraziamenti

Indice

Sommario iii
Ringraziamenti iv
Indice v
Elenco delle figure vii
Elenco delle tabelle viii
1 Introduzione ..... 1.
1.1 L'azienda ..... 1.
1.2 Il progetto ..... 1.
1.3 Obiettivi dello stage ..... 2.
1.4 Perimetro e vincoli ..... 2.
1.5 Organizzazione dello stage ..... 2.
1.6 Organizzazione del testo ..... 3.
2 Processi, metodologie e strumenti ..... 5.
2.1 Pianificazione e ripartizione delle attività ..... 5.
2.2 Requisiti e obiettivi ..... 7.
2.2.1 Requisiti Obbligatori (O) ..... 7.
2.2.2 Requisiti Desiderabili (D) ..... 7.
2.2.3 Requisiti Facoltativi (F) ..... 8.
2.3 Modello Agile e gestione iterativa ..... 8.
2.4 Issue Tracking e Controllo di Versione ..... 9.
2.5 Analisi preventiva dei rischi ..... 9.
3 Analisi dei requisiti ..... 13.
3.1 Analisi degli utenti ..... 13.
3.2 Casi d'uso ..... 13.
3.2.1 Elenco dei casi d'uso ..... 14.
3.2.2 Descrizione dettagliata dei casi d'uso ..... 14.
3.3 Requisiti del sistema e stato di completamento ..... 26.
3.3.1 Requisiti funzionali e stato di attuazione ..... 27.
3.3.2 Requisiti non funzionali, di qualità e vincolo ..... 28.
3.3.3 Conclusione sulla soddisfazione dei requisiti ..... 29.
4 Tecnologie utilizzate ..... 31.
4.1 Piattaforma hardware ..... 31.
4.2 Linguaggio C++ ..... 32.
4.3 PlatformIO ..... 33.
4.4 LVGL e interfaccia grafica ..... 34.
4.5 Connettività e protocolli ..... 35.
4.6 Persistenza e gestione dei dati ..... 36.
4.7 Controllo di versione e documentazione ..... 36.
4.7.1 Gitea ..... 36.

4.7.2 Typst per la redazione della documentazione ..... 36.
4.8 Riepilogo dello stack tecnologico ..... 37.

Elenco delle figure

Figura 1 Logo di Spazio Dev S.r.l. ..... 1.

Elenco delle tabelle

1 Introduzione

La gestione operativa delle organizzazioni e il monitoraggio in tempo reale delle attività aziendali richiedono spesso di coordinare flussi informativi complessi, tradizionalmente confinati all'interno di piattaforme software gestionali o pannelli web di controllo. Quando sorge l'esigenza di rendere questi indicatori immediatamente accessibili in contesti fisici distribuiti — come uffici, postazioni di accoglienza o spazi espositivi — l'utilizzo di soluzioni hardware dedicate consente di superare l'attrito dei sistemi tradizionali, offrendo informazioni visive immediate senza la necessità di postazioni informatiche complesse.

Questo elaborato presenta il progetto di stage svolto presso l'azienda **Spazio Dev S.r.l.**, che prevede la realizzazione e lo sviluppo di un prototipo di dispositivo interattivo basato su microcontrollore e display, progettato per integrarsi direttamente con la piattaforma CRM **RelAi**. Il progetto prende come riferimento i requisiti operativi e l'architettura aziendale, ponendosi l'obiettivo di verificare la fattibilità e l'efficacia di un'interfaccia *embedded* per la visualizzazione in tempo reale di dati chiave quali iscrizioni, metriche di eventi e codici di servizio attraverso un prototipo funzionante.

1.1 L'azienda

Spazio Dev S.r.l. opera nel settore dello sviluppo software e della consulenza informatica, offrendo soluzioni tecnologiche avanzate per la gestione aziendale e dei flussi operativi. Il progetto di stage si inserisce nel contesto dell'ecosistema software aziendale, integrandosi con la piattaforma CRM RelAi per estendere le funzionalità di monitoraggio e interazione visiva direttamente su dispositivi *embedded* installati presso gli utenti finali

Figura 1: Logo di Spazio Dev S.r.l.

.

1.2 Il progetto

L'attività di stage riguarda la progettazione e lo sviluppo di una piattaforma firmware per microcontrollori ESP32 dotati di display. I dispositivi realizzati devono essere in grado di comunicare con il CRM RelAi e di presentare informazioni aggiornate in tempo reale mediante un'interfaccia grafica semplice e

facilmente leggibile.
La soluzione proposta prevede l'utilizzo di pannelli informativi intelligenti capaci di visualizzare differenti tipologie di contenuto, tra cui codici QR, informazioni relative agli eventi e dati statistici riguardanti organizzazioni e partecipanti. Il progetto comprende inoltre tutti gli aspetti necessari alla configurazione, all'integrazione e alla gestione operativa dei dispositivi.

1.3 Obiettivi dello stage

Lo stage si pone l'obiettivo di sviluppare una soluzione affidabile e facilmente utilizzabile, in grado di collegare il mondo dei sistemi gestionali aziendali con quello dei dispositivi embedded.
Le attività previste comprendono:

- l'analisi dei requisiti funzionali e tecnici del sistema;
- lo studio della piattaforma hardware ESP32 e delle tecnologie software necessarie;
- la progettazione dell'architettura del sistema;
- lo sviluppo dell'interfaccia grafica e dei meccanismi di comunicazione con il CRM;
- la verifica del corretto funzionamento della soluzione mediante attività di test;
- la produzione della documentazione tecnica relativa al progetto.

Nel corso delle prime settimane di attività è stata svolta un'intensa fase di analisi e studio preliminare, finalizzata alla comprensione della piattaforma hardware, delle librerie software e delle modalità di integrazione con il sistema RelAi. Tale fase costituisce la base per le successive attività di sviluppo e validazione previste dal piano di lavoro.

1.4 Perimetro e vincoli

Lo sviluppo su microcontrollore ESP32 impone vincoli rigorosi in termini di risorse computazionali e gestione della memoria. Il sistema deve inoltre garantire un livello elevato di robustezza operativa in esercizio, gestendo scenari di disconnessione della rete Wi-Fi o interruzioni temporanee della comunicazione con le API di RelAi attraverso meccanismi di cache locale, riconnessione automatica e controllo tramite *watchdog*.

1.5 Organizzazione dello stage

Il piano di attività ha una durata complessiva di 300 ore, suddivise indicati-

vamente in otto settimane (dal 21 settembre al 20 novembre 2026). Le prime settimane sono dedicate all'analisi preliminare, allo studio della piattaforma hardware ESP32 e delle librerie grafiche, per poi procedere con lo sviluppo incrementale del provisioning di rete, del client API, del rendering dei pannelli e delle funzioni di diagnostica e test finali.

1.6 Organizzazione del testo

- Il **secondo capitolo** descrive i processi e le metodologie di sviluppo adottati.
- Il **terzo capitolo** presenta l'analisi degli utenti, i casi d'uso e i requisiti del sistema.
- Il **quarto capitolo** descrive le tecnologie hardware e software utilizzate per lo sviluppo del prototipo.
- Il **quinto capitolo** tratta la progettazione architetturale e la codifica del firmware ESP32.
- Il **sesto capitolo** descrive le attività di verifica, validazione e testing sul campo.
- Il **settimo capitolo** riassume i risultati conseguiti e le conclusioni.
- L'**ottavo capitolo** raccoglie la bibliografia e i riferimenti sitografici.

Gli acronimi sono sciolti alla prima occorrenza e i termini tecnici o stranieri sono evidenziati in *corsivo*.

4.

2 Processi, metodologie e strumenti

Il presente capitolo approfondisce il quadro metodologico e gli strumenti adottati nel corso delle 300 ore di stage svolto presso **Spazio Dev S.r.l.** (con sede a Tombolo, Padova). L'azienda è una realtà fortemente orientata allo sviluppo software di qualità e all'innovazione tecnologica, che promuove un ambiente collaborativo basato sulla condivisione delle competenze e sul miglioramento continuo.
Per affrontare la complessità dello sviluppo *embedded* e garantire standard qualitativi coerenti con le esigenze aziendali, l'attività è stata pianificata seguendo un approccio strutturato. L'integrazione del prototipo basato su microcontrollore ESP32 con la piattaforma CRM RelAi ha richiesto non solo competenze tecniche di programmazione e interfacciamento con API, ma anche una rigorosa aderenza ai flussi di lavoro interni, garantendo il rispetto delle tempistiche e un costante allineamento sugli obiettivi strategici del prodotto.

2.1 Pianificazione e ripartizione delle attività

Il percorso di stage è stato distribuito nell'arco di otto settimane (sette settimane da 40 ore e una settimana finale da 20 ore), per un totale di 300 ore complessive. Le attività sono state pianificate bilanciando la fase di studio iniziale, lo sviluppo del firmware e la validazione finale. La progressione temporale si articola nelle seguenti fasi:

- Prima Settimana - Analisi iniziale e setup (40 ore): onboarding sul progetto, configurazione dell'ambiente di lavoro e del repository, raccolta dei requisiti con il tutor e comprensione dei casi d'uso principali.
- Seconda Settimana - Studio dell'ESP32, del display e delle librerie grafiche (40 ore): studio dell'architettura del microcontrollore e del display, configurazione della toolchain e valutazione delle librerie grafiche e per i codici QR.
- Terza Settimana - Configurazione iniziale del dispositivo: provisioning WiFi e reset (40 ore): implementazione dell'avvio in modalità access point con portale di configurazione, salvataggio persistente e reset alle impostazioni di fabbrica.
- Quarta Settimana - Abbinamento a RelAi, client API e cache locale (40 ore): sviluppo della procedura di abbinamento al CRM, del client verso le API, della gestione degli errori e della cache locale dell'ultimo dato valido.
- Quinta Settimana - Pannello codice QR e gestione delle schermate (40 ore): generazione e rendering del codice QR e realizzazione della macchina a stati per la gestione delle schermate di stato e di errore.

- Sesta Settimana - Pannelli contatori e prenotazioni, rotazione automatica (40 ore): sviluppo dei pannelli relativi a iscritti, partecipanti e prenotazioni, con rotazione automatica e rifinitura grafica.
- Settima Settimana - Configurazione remota, diagnostica e robustezza (40 ore): recupero della configurazione da RelAi, esposizione della pagina di diagnostica locale e introduzione di meccanismi di robustezza come il *watchdog* e la riconnessione automatica.
- Ottava Settimana - Test, rifinitura e confronto finale (20 ore): esecuzione dei test funzionali e di durata prolungata, correzione dei bug, stesura della relazione finale e preparazione della demo conclusiva.

A supporto della pianificazione temporale, la ripartizione analitica delle ore in base all'impegno richiesto dai singoli moduli è definita come segue:

- **18 ore:** Onboarding, analisi dei requisiti e definizione dei pannelli informativi con il tutor aziendale.
- **14 ore:** Setup dell'ambiente di sviluppo (toolchain ESP32, repository, strumenti di flashing e di test).
- **30 ore:** Studio della piattaforma ESP32, del display e delle librerie grafiche e per la generazione dei codici QR.
- **22 ore:** Progettazione dei layout dei pannelli e analisi delle API di RelAi e dei dati da visualizzare.
- **38 ore:** Provisioning della rete Wi-Fi con portale di configurazione, salvataggio persistente delle impostazioni e reset del dispositivo.
- **28 ore:** Abbinamento del dispositivo a RelAi, gestione delle credenziali di accesso e del ciclo di vita dell'associazione.
- **28 ore:** Client verso le API, aggiornamento periodico dei dati, cache locale e sincronizzazione dell'ora.
- **45 ore:** Rendering dei pannelli sul display (codice QR, contatori e prenotazioni evento) e gestione delle schermate di stato.
- **18 ore:** Configurazione remota dei pannelli da RelAi e pagina di diagnostica locale del dispositivo.
- **14 ore:** Robustezza del dispositivo (watchdog, riconnessione, riavvio automatico) e predisposizione dell'aggiornamento del firmware.
- **25 ore:** Test funzionali e di durata sul dispositivo, bug fixing e miglioramenti grafici.
- **20 ore:** Documentazione tecnica, relazione finale e preparazione della presentazione.

2.2 Requisiti e obiettivi

Per dimensionare correttamente il progetto rispetto alle trecento ore di stage e monitorare il grado di completamento delle funzionalità, gli obiettivi e i requisiti sono stati classificati in tre livelli gerarchici:

2.2.1 Requisiti Obbligatori (O)

- **O01 (Configurazione e Provisioning):** Realizzare la procedura di configurazione iniziale del dispositivo, comprendendo il provisioning della rete Wi-Fi tramite portale di configurazione, il salvataggio persistente delle impostazioni e la funzione di reset alle impostazioni di fabbrica.
- **O02 (Integrazione CRM e API):** Sviluppare l'abbinamento del dispositivo al CRM RelAi e implementare il client verso le API aziendali, garantendo l'aggiornamento periodico dei dati e la cache locale dell'ultimo valore valido.
- **O03 (Visualizzazione e Gestione Stati):** Visualizzare sul display il codice QR, il contatore degli iscritti, il contatore dei partecipanti e le prenotazioni dell'evento, implementando schermate dedicate per gestire gli stati di errore e l'assenza di rete.
- **O04 (Robustezza e Affidabilità):** Garantire la stabilità operativa e l'affidabilità del dispositivo in condizioni d'uso prolungate, sottoponendo il sistema a rigorosi test funzionali e di stress prima della conclusione dello stage.

2.2.2 Requisiti Desiderabili (D)

- **D01 (Configurazione Remota):** Permettere la configurazione remota da RelAi dei pannelli mostrati e dell'intervallo di aggiornamento, evitando la necessità di riflashare il dispositivo.
- **D02 (Diagnostica Locale):** Esporre una pagina di diagnostica locale servita direttamente dal dispositivo, utile per monitorare lo stato della rete, l'ultima sincronizzazione, la versione del firmware e per attivare azioni di riavvio e reset.
- **D03 (Rotazione Automatica):** Introdurre la rotazione automatica tra più pannelli attivi configurando intervalli di durata personalizzabili.
- **D04 (Aggiornamento Firmware OTA):** Predisporre meccanismi di aggiornamento del firmware over-the-air (OTA) con controllo della versione disponibile.

2.2.3 Requisiti Facoltativi (F)

- **F01 (Aggiornamento Near-Real-Time):** Sostituire o integrare l'aggiornamento periodico con un flusso di dati *near-real-time* basato su protocolli come WebSocket o MQTT.
- **F02 (Check-in Partecipanti):** Gestire la registrazione del check-in dei partecipanti a un evento mostrando sul pannello un codice QR identificativo, che il socio può inquadrare con il proprio smartphone per completare la registrazione lato server su RelAi.
- **F03 (Supporto Multi-display e Temi):** Estendere il supporto ad altri modelli di display e introdurre la personalizzazione grafica dei pannelli tramite temi e loghi dell'organizzazione.

2.3 Modello Agile e gestione iterativa

L'organizzazione del lavoro ha ripreso i principi chiave delle metodologie **Agile**, suddividendo lo sviluppo complessivo in cicli iterativi brevi orientati al rilascio incrementale delle funzionalità. Questo approccio ha permesso di validare progressivamente i singoli componenti del sistema — dalla connettività Wi-Fi al rendering grafico su display — riducendo i rischi di regressione e consentendo correzioni tempestive in corso d'opera.
La comunicazione e la collaborazione con il tutor aziendale (Matteo Forzan) sono state strutturate attraverso incontri periodici fissati con cadenza regolare (due o tre volte a settimana), articolati secondo una precisa suddivisione dei ruoli operativi:

- **Primo incontro (inizio settimana):** dedicato alla pianificazione e revisione dei task settimanali, con l'individuazione dei pannelli o delle funzionalità da realizzare, la stima dell'impegno richiesto e l'assegnazione delle priorità operative in base agli obiettivi dello stage.
- **Incontro intermedio (metà settimana):** volto all'analisi tecnica e alla risoluzione di dubbi o criticità emerse durante lo sviluppo, con particolare focus sui colli di bottiglia legati alla connettività del dispositivo e al rendering sul display.
- **Ultimo incontro (fine settimana):** finalizzato alla verifica delle mansioni svolte attraverso test empirici e prove dirette dei pannelli sul dispositivo fisico, alla raccolta di feedback costruttivi e alla validazione dello stato di avanzamento rispetto al cronoprogramma.

In aggiunta ai momenti di confronto, il codice sorgente del firmware ESP32 è stato sottoposto a regolari sessioni di **code review** con il tutor. Tale pratica ha garantito un costante controllo sulla qualità del codice, sulla leggibilità, sul rispetto delle convenzioni di stile e sulla correttezza funzionale prima di procedere all'integrazione definitiva.

2.4 Issue Tracking e Controllo di Versione

Per il monitoraggio delle attività, la pianificazione operativa e la gestione centralizzata del codice sorgente, sono stati adottati strumenti open-source moderni ed efficenti, perfettamente integrati nei flussi di lavoro aziendali:

- **Controllo di versione:** La gestione del codice è stata affidata a **Gitea**, che ha ospitato i repository del progetto. Il flusso di lavoro è stato modellato attraverso l'uso di branch dedicati per singola funzionalità o correzione (`feature/`), l'apertura sistematica di *pull request* mirate e la supervisione tramite code review. Questa organizzazione ha permesso di isolare le modifiche sperimentali, preservare la stabilità del ramo principale e mantenere una cronologia pulita e tracciabile di tutte le evoluzioni del software.
- **Issue Tracking:** La tracciabilità e la pianificazione delle attività giornaliere e settimanali sono state gestite mediante la piattaforma **Plane**. Ogni requisito funzionale o correzione di bug è stato censito sotto forma di *issue*, suddiviso in task granulari e associato a specifiche etichette di priorità. Questo strumento ha consentito di monitorare visivamente lo stato di avanzamento delle 300 ore di stage, offrendo un riscontro immediato sul carico di lavoro corrente e facilitando la rendicontazione delle attività svolte.

2.5 Analisi preventiva dei rischi

Prima dell'avvio dello sviluppo, sono stati identificati i principali fattori di rischio associati all'hardware e all'integrazione software, definendo contestualmente le relative strategie di mitigazione:

- **Complessità di integrazione hardware-software e concorrenza:**
- **Descrizione:** La gestione simultanea del rendering grafico avanzato sul display e delle comunicazioni di rete asincrone con le API del CRM genera colli di bottiglia o blocchi del microcontrollore.

- **Mitigazione:** Adozione di un'architettura software strutturata in task paralleli, separando la logica di rete dalle routine di aggiornamento grafico e sincronizzando l'accesso alle risorse condivise in modo sicuro.
- **Probabilità:** Media
- **Impatto:** Alto

- **Esaurimento della memoria RAM e instabilità di rete:**
- **Descrizione:** Si verificano errori a runtime di tipo *out of memory* dovuti ai limiti fisici della memoria interna dell'ESP32, accompagnati da interruzioni operative prolungate in caso di instabilità o assenza della connessione Wi-Fi.
- **Mitigazione:** Ottimizzazione dell'uso della memoria RAM attraverso il rilascio mirato dei buffer grafici non utilizzati e implementazione della cache locale per garantire la continuità d'uso in caso di disconnessione temporanea della rete.
- **Probabilità:** Media
- **Impatto:** Medio-Alto

- **Evoluzione o instabilità delle API del CRM:**
- **Descrizione:** Modifiche impreviste agli endpoint o ai contratti delle API del sistema CRM esterno compromettono la corretta ricezione e trasmissione dei dati.
- **Mitigazione:** Astrazione del layer di comunicazione di rete tramite interfacce modulari e implementazione di robusti controlli di validazione e gestione delle eccezioni sui payload in ingresso e in uscita.
- **Probabilità:** Bassa
- **Impatto:** Alto

- **Prestazioni e fluidità dell'interfaccia grafica:**
- **Descrizione:** La visualizzazione di informazioni aggiornate in tempo reale e la gestione delle animazioni dell'interfaccia provocano rallentamenti o riducono la reattività del sistema sui dispositivi ESP32.

- **Mitigazione:** Ottimizzazione dei componenti grafici, riduzione degli aggiornamenti non necessari e monitoraggio dell'utilizzo delle risorse durante le attività di sviluppo e test.
- **Probabilità:** Media
- **Impatto:** Medio
- **Disallineamento dei requisiti funzionali per i pannelli:**
- **Descrizione:** Interpretazioni errate delle specifiche dei pannelli informativi o delle modalità di interazione con i display vincolati da risorse limitate comportano lo sviluppo di funzionalità non conformi alle reali esigenze operative dell'organizzazione cliente.
- **Mitigazione:** Gestione costante attraverso gli incontri periodici e regolari di allineamento e le sessioni di revisione con il tutor aziendale.
- **Probabilità:** Bassa
- **Impatto:** Medio

- **Sforamento dei tempi di sviluppo:**
- **Descrizione:** La curva di apprendimento legata alla programmazione embedded su ESP32, unita a criticità impreviste nell'interfacciamento con le API di RelAi, allunga i tempi di completamento delle attività previste dal cronoprogramma.
- **Mitigazione:** Pianificazione incrementale delle attività, definizione delle priorità in funzione dei requisiti obbligatori e monitoraggio costante dell'avanzamento mediante incontri periodici con il tutor aziendale, così da individuare tempestivamente eventuali ritardi.
- **Probabilità:** Media
- **Impatto:** Medio-Alto

3 Analisi dei requisiti

Il presente capitolo è dedicato all'analisi dei requisiti del sistema, attività essenziale per definire gli obiettivi funzionali e i vincoli progettuali della soluzione. Vengono illustrati gli attori coinvolti, i casi d'uso e i requisiti funzionali e non funzionali individuati durante la fase di studio. Le informazioni raccolte costituiscono il punto di partenza per le successive attività di progettazione e implementazione del sistema.

3.1 Analisi degli utenti

Il sistema sviluppato nel contesto dello stage consiste in un dispositivo basato su ESP32, dotato di display e integrato con il CRM RelAi. Il suo scopo è mostrare in tempo reale informazioni relative a organizzazioni ed eventi, quali codici QR, contatori degli iscritti e dei partecipanti, prenotazioni e dati di stato del dispositivo.
L'utilizzo del sistema coinvolge un unico attore principale, identificato nell'utente dell'organizzazione cliente presso la quale il dispositivo viene installato. Tale soggetto è responsabile della configurazione iniziale del pannello, che comprende il collegamento alla rete Wi-Fi tramite portale captive e la gestione delle principali impostazioni operative del dispositivo trammite CRM RelAi.
Una volta completata la fase di configurazione, l'utente interagisce con il sistema attraverso la visualizzazione ciclica o manuale dei pannelli informativi a display e l'utilizzo di un menu diagnostico touch locale per operazioni di manutenzione (riavvio, reset, condivisione QR). Parallelamente, il sistema dialoga con il cloud per ricevere aggiornamenti in tempo reale tramite WebSocket.
Poiché il progetto è destinato a contesti operativi nei quali gli utilizzatori potrebbero non possedere particolari competenze tecniche, l'interfaccia e i flussi di interazione sono stati progettati privilegiando chiarezza, semplicità di utilizzo e rapidità delle operazioni di configurazione e gestione. Tale approccio consente di ridurre la complessità percepita e di facilitare l'adozione del dispositivo presso le organizzazioni clienti.

3.2 Casi d'uso

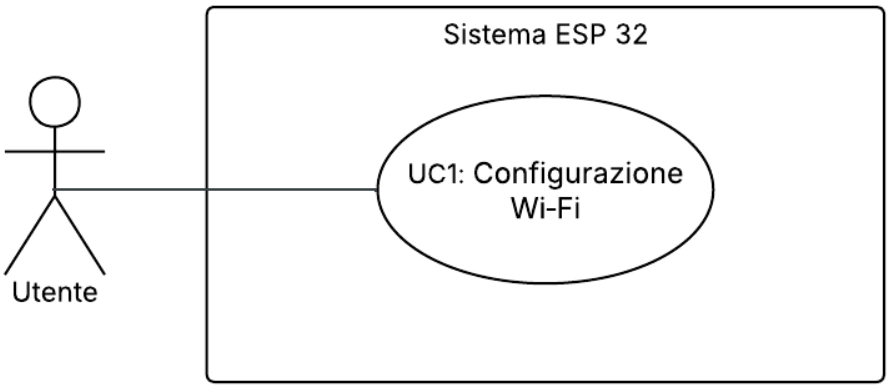
Per lo studio dei casi di utilizzo del prodotto sono stati definiti i diagrammi e le relative specifiche funzionali. I diagrammi dei casi d'uso (*Use Case Diagram*) sono diagrammi di tipo *UML* dedicati alla descrizione delle funzioni o servizi offerti dal sistema, così come sono percepiti dagli attori che interagiscono con esso.

3.2.1 Elenco dei casi d'uso

- **UC1** – Configurazione Wi-Fi
  - **UC1.1** – Nessuna rete disponibile
  - **UC1.2** – Credenziali non valide
- **UC2** – Consultazione diagnostica locale del dispositivo
- **UC3** – Consultazione stato e telemetria remota tramite CRM RelAi
- **UC4** – Condivisione Wi-Fi tramite QR Code
- **UC5** – Riavvio locale del dispositivo
- **UC6** – Riavvio remoto del dispositivo
- **UC7** – Reset di sistema locale
- **UC8** – Reset di sistema remoto
- **UC9** – Sincronizzazione e gestione remota tramite CRM RelAi
  - **UC9.1** – Gestione del fallback in assenza di connettività
- **UC10** – Navigazione tra le schermate informative
- **UC11** – Visualizzazione dei dati nei pannelli informativi a display

3.2.2 Descrizione dettagliata dei casi d'uso

UC 1: Configurazione della connessione Wi-Fi



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- È disponibile almeno una rete Wi-Fi nelle vicinanze.

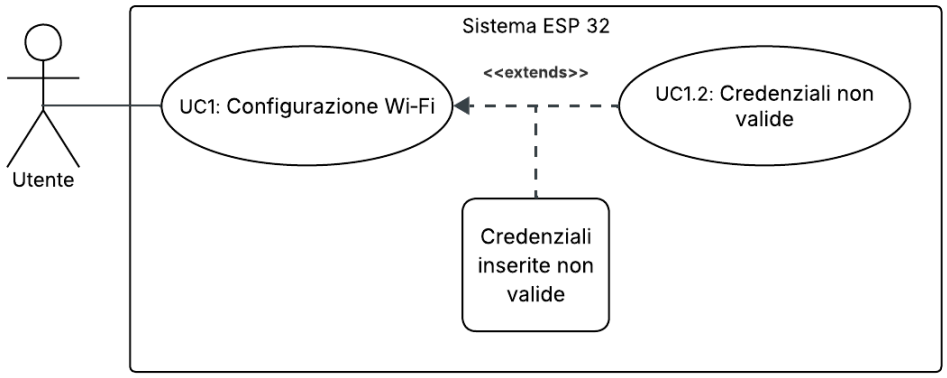
Trigger: L'utente desidera configurare o aggiornare i parametri di accesso alla rete Wi-Fi del dispositivo.
Descrizione:

1. L'utente si connette all'Access Point generato dal dispositivo.
2. Il sistema reindirizza automaticamente l'utente alla pagina di configurazione.
3. Il sistema esegue la scansione delle reti Wi-Fi disponibili.
4. L'utente seleziona la rete desiderata.
5. L'utente inserisce la password della rete selezionata.
6. L'utente conferma la configurazione.
7. Il sistema verifica i dati ricevuti.
8. Il sistema salva le credenziali nella memoria persistente.
9. Il sistema effettua la connessione alla rete Wi-Fi selezionata.
10. Il sistema conferma l'avvenuta configurazione.

Postcondizioni:

- Le credenziali della rete Wi-Fi sono memorizzate nella memoria persistente del dispositivo.
- Il dispositivo risulta connesso alla rete selezionata.
- Il dispositivo è pronto per comunicare con le API del CRM RelAi.

UC 1.1: Nessuna rete disponibile



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso con il Captive Portal attivo
- L'utente è connesso alla rete generata dal dispositivo.

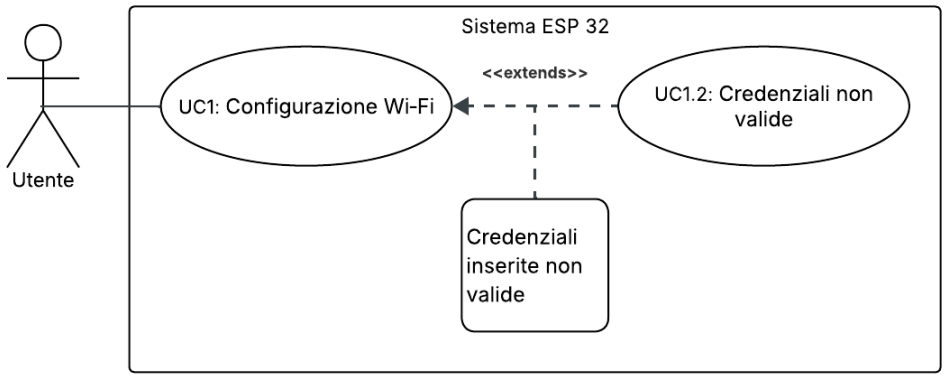
Trigger: Avvio della scansione delle reti Wi-Fi senza la presenza di punti d'accesso validi nel raggio d'azione.
Descrizione:

1. L'utente accede alla pagina di configurazione Wi-Fi.

2. Il sistema avvia la scansione delle reti disponibili.
3. La scansione non restituisce alcuna rete utilizzabile o va in timeout.
4. Il sistema restituisce l'elenco vuoto e un feedback visivo d'errore all'interfaccia web.
5. L'utente richiede una nuova scansione.

Postcondizioni:
- Nessuna configurazione Wi-Fi viene salvata.
- Il Captive Portal e l'Access Point rimangono disponibili per un nuovo tentativo.

UC 1.2: Credenziali non valide



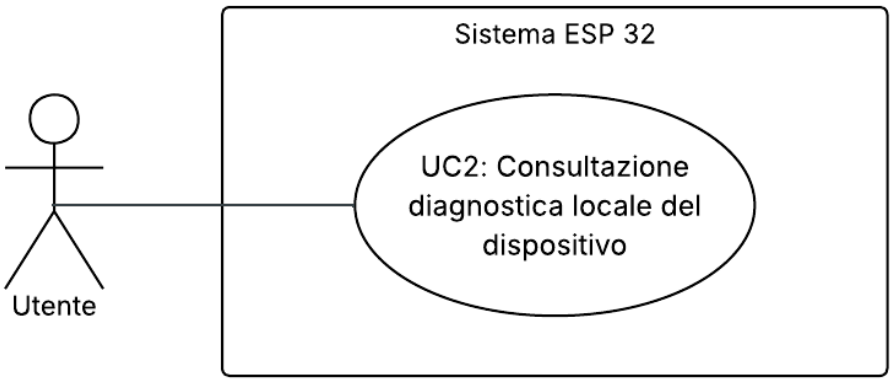
Attori principali: Utente
Precondizioni:
- È stata selezionata una rete Wi-Fi.
- Sono stati inseriti dati errati.

Trigger: Tentativo di connessione a una rete Wi-Fi con parametri sbagliati.
Descrizione:
1. L'utente conferma la configurazione inserendo la password.
2. Il sistema tenta di collegarsi alla rete Wi-Fi per un intervallo massimo di timeout.
3. Il tentativo di associazione fallisce (codice di errore Wi-Fi).
4. Il sistema intercetta il fallimento, interrompe la connessione STA e ripristina la stabilità dell'Access Point.
5. Il sistema restituisce un messaggio di errore JSON all'interfaccia di provisioning.
6. L'utente corregge i parametri e ripete la procedura.

Postcondizioni:

- Nessuna credenziale errata viene salvata come configurazione stabile.
- Il portale di configurazione rimane attivo per consentire la correzione dei dati.

UC 2: Consultazione diagnostica locale del dispositivo



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- L'interfaccia grafica è operativo.

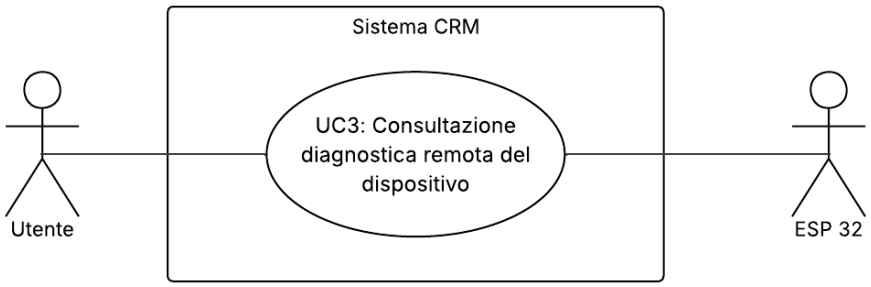
Trigger: L'utente seleziona il pulsante «Stato Sistema» dal menu touch locale.
Descrizione:

1. L'utente preme il pulsante di accesso allo stato dal menu touch del display.
2. Il sistema interroga i moduli di runtime e raccoglie i parametri correnti (SSID della rete connessa, stato della connessione, versione del firmware e tempo trascorso *uptime* dall'ultima disconnessione).
3. Il sistema apre una *message box* contenente le informazioni aggregate.
4. L'utente prende visione dei dati diagnostici e chiude la finestra modale tramite il pulsante di chiusura dedicato, ripristinando la schermata principale.

Postcondizioni:

- L'utente è informato sullo stato di salute e connettività locale del dispositivo.
- Il sistema riprende la normale rotazione o visualizzazione dei pannelli alla chiusura del modale.

UC 3: Consultazione stato e telemetria remota tramite CRM RelAi



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

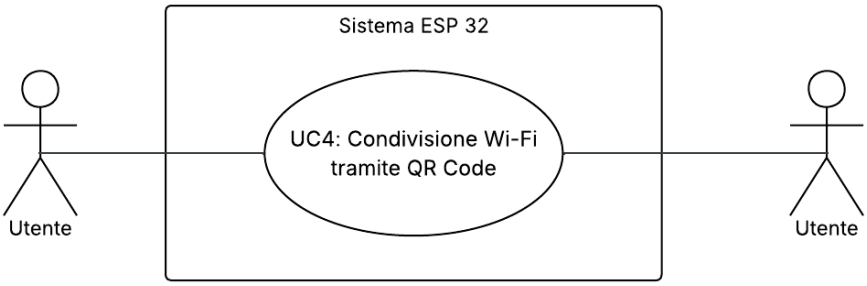
Trigger: L'utente desidera verificare lo stato di salute, la connettività o le metriche di un dispositivo dal pannello di controllo cloud.
Descrizione:

1. L'utente accede alla sezione di monitoraggio dei dispositivi all'interno della piattaforma CRM RelAi.
2. Il sistema cloud interroga i dati di telemetria più recenti memorizzati dal dispositivo (tra cui versione software, *uptime* e RSSI del Wi-Fi).
3. L'interfaccia della piattaforma CRM elabora le informazioni e mostra all'utente la panoramica aggiornata sullo stato di salute e connettività del display.

Postcondizioni:

- L'utente ottiene una visione centralizzata e aggiornata dello stato operativo del dispositivo da remoto.
- La piattaforma cloud mantiene traccia delle metriche telemetriche ricevute.

UC 4: Condivisione Wi-Fi tramite QR Code



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso ed è connesso a una rete Wi-Fi.

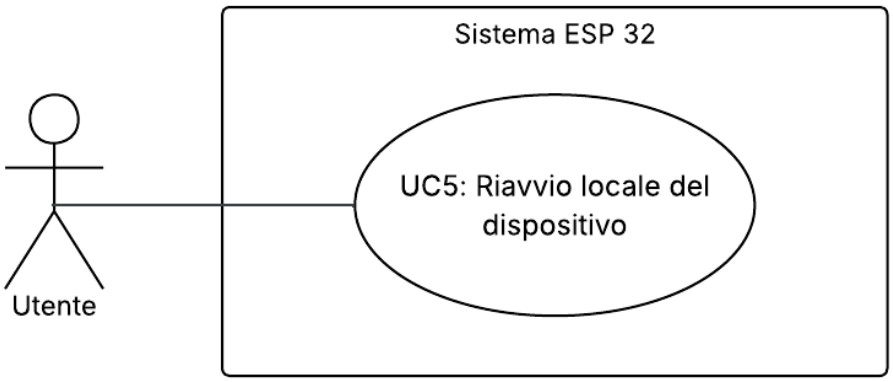
Trigger: L'utente seleziona il pulsante «QR Rete» dal menu di diagnostica locale per contividere i dati di accesso alla rete .
Descrizione:

1. L'utente accede al menu sul display e seleziona la condivisione della rete.
2. Il sistema recupera l'SSID e la chiave di rete.
3. Il sistema formatta la stringa secondo lo standard di connessione mobile.
4. Il sistema genera dinamicamente un codice QR e lo renderizza a schermo all'interno di una *message box* edicata.
5. Un dispositivo esterno scansiona il codice QR per connettersi rapidamnte alla medesima rete.
6. L'utente chiude il modale al termine dell'operazione.

Postcondizioni:

- Il codice QR con le credenziali correnti risulta visibile a schermo.
- Alla chiusura del modale, viene ripristinata la visualizzazione operativa standard e riattivato il timer di rotazione.

UC 5: Riavvio locale del dispositivo



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- L'interfaccia grafica è operativo.

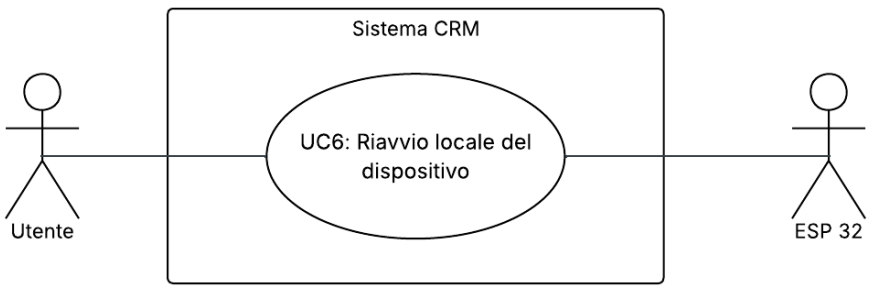
Trigger: L'utente seleziona il tasto «Riavvio ESP32» sul display.
Descrizione:

1. L'utente seleziona l'opzione di riavvio dal menu locale touch.
2. Il sistema avvia un timer temporizzato di sicurezza per la chiusura ordinata dei task.
3. Il sistema esegue il comando software di riavvio.
4. Il microcontrollore si riavvia ricaricando il firmware e ripristinando le periferiche e le connessioni di rete.

Postcondizioni:

- Il dispositivo viene riavviato pulitamente preservando le configurazioni e le preferenze salvate in memoria persistente NVS.

UC 6: Riavvio remoto del dispositivo



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

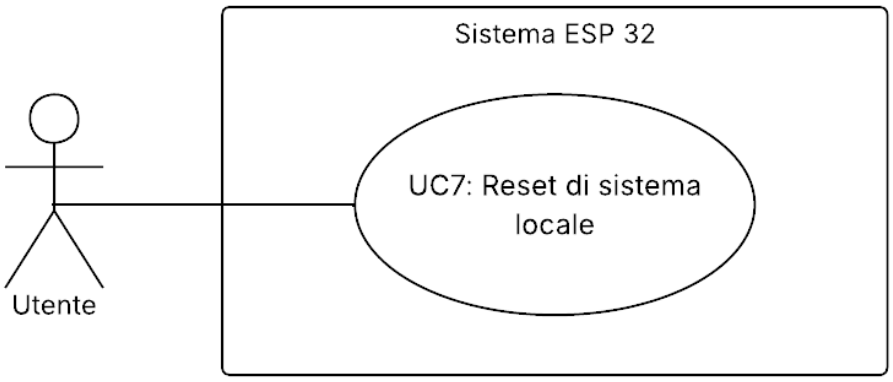
Trigger: L'utente invia un comando di riavvio remoto dalla piattaforma cloud CRM RelAi.
Descrizione:

1. Il sistema riceve un payload JSON tramite WebSocket contenente il comando di riavvio.
2. Il sistema elabora il comando remoto e avvia la procedura di chiusura delle attività in corso.
3. Il microcontrollore esegue il comando di riavvio software.
4. Il dispositivo si riavvia da remoto ricaricando il firmware.

Postcondizioni:

- Il dispositivo viene riavviato da remoto preservando le configurazioni salvate in memoria persistente NVS.

UC 7: Reset di sistema locale



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è acceso e operativo.

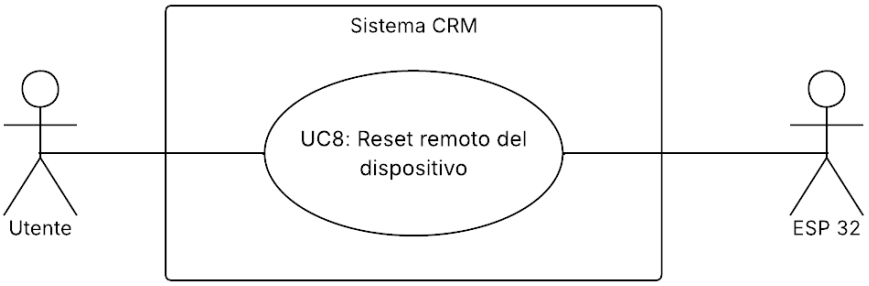
Trigger: L'utente seleziona il tasto «Reset ESP32» dal display.
Descrizione:

1. L'utente seleziona l'opzione di reset di fabbrica dal menu locale.
2. Il sistema avvia la procedura di cancellazione dei namespace di configurazione nella memoria flash.
3. Il sistema disconnette le interfacce di rete e azzera le preferenze salvate.
4. Il sistema esegue il riavvio automatico del microcontrollore.
5. Al nuovo avvio, il dispositivo si presenta nello stato iniziale di fabbrica con l'Access Point di provisioning attivo.

Postcondizioni:

- Tutti i dati salvati, le preferenze di rete e la cache locale risultano completamente cancellati.
- Il dispositivo torna allo stato di configurazione iniziale pronto per una nuova associazione.

UC 8: Reset di sistema remoto



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: L'utente invia un comando di reset di fabbrica dalla piattaforma cloud CRM RelAi.
Descrizione:

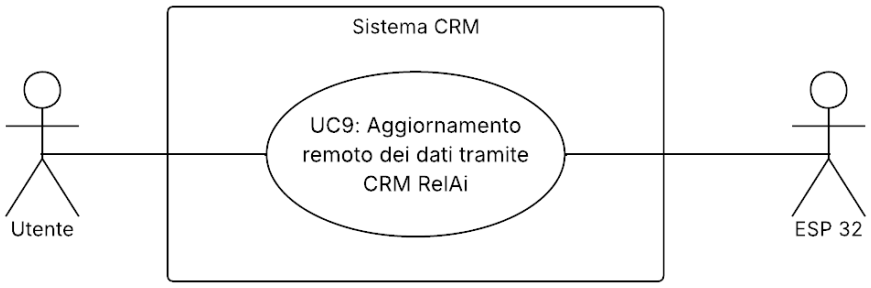
1. Il sistema riceve un payload JSON tramite WebSocket contenente il comando di reset.
2. Il sistema avvia la procedura di cancellazione remota dei dati nei namespace di configurazione presenti nella memoria flash.
3. Il sistema ripulisce le preferenze e interrompe le interfacce di rete.

4. Il sistema esegue il riavvio automatico del dispositivo portandolo allo stato iniziale di fabbrica.

Postcondizioni:

- Tutte le preferenze e i dati memorizzati in precedenza risultano cancellati da remoto.
- Il dispositivo torna allo stato di configurazione iniziale con l'Access Point di provisioning attivo.

UC 9: Aggiornamento remoto dei dati tramite CRM RelAi



Attori principali: Utente

Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 è connesso a Internet tramite Wi-Fi.
- Il dispositivo ESP32 e connesso al CRM RelAi .

Trigger: Ricezione di un evento di aggiornamento in tempo reale sul canale WebSocket.

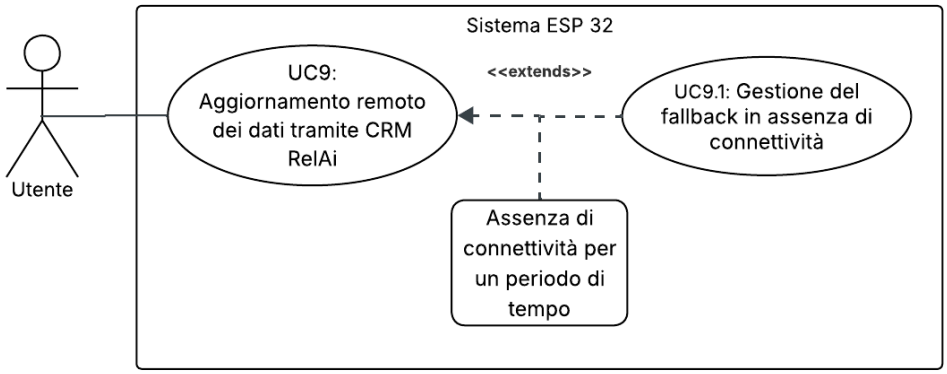
Descrizione:

1. Il sistema riceve i dati di configurazione aggiornati e la lista aggiornata degli eventi dal server.
2. Il modulo di gestione dati elabora il payload JSON ricevuto e verifica se vi sono variazioni rispetto alla cache corrente.
3. Se i dati differiscono, il sistema salva i nuovi eventi nella memoria persistente NVS.
4. Il sistema invia un segnale interno di aggiornamento per ricostruire dinamicamente le schermate informative.

Postcondizioni:

- Le schermate informative, i dati degli eventi e i parametri di rotazione si aggiornano dinamicamente in base alle direttive del CRM RelAi.

UC 9.1: Gestione del fallback in assenza di connettività



Attori principali: Utente
Precondizioni:

- Il dispositivo si trova in uno stato di assenza temporanea di connessione della rete o il server CRM RelAi risulta irragiungibile.

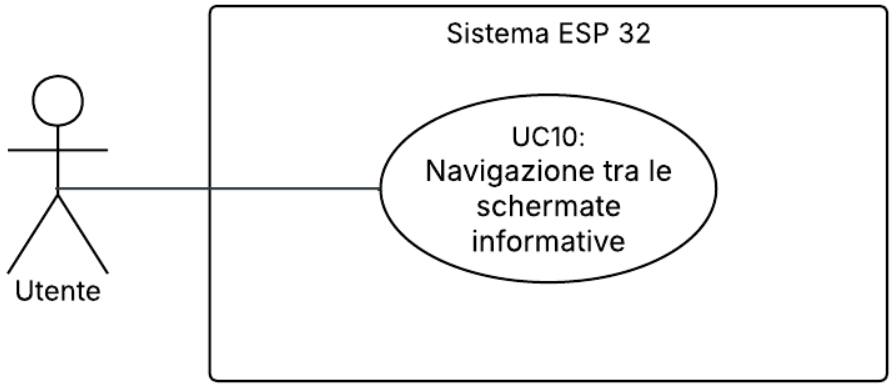
Trigger: Fallimento di una richiesta di aggiornamento dei dati in tempo reale.
Descrizione:

1. Il sistema rileva il fallimento della comunicazione di rete o l'assenza prolungata di connettività.
2. Il gestore delle API intercetta l'eccezione bloccando i tentativi non necessari e prevenendo la frammentazione della memoria RAM.
3. Il sistema legge in modo trasparente i dati precedentemente memorizzati nella cache persistente locale.
4. L'interfaccia grafica continua a mostrare l'ultimo stato valido degli eventi senza causare blocchi o riavvii anomali del display.
5. Il sistema avvia una procedura di tentativi schedulati di riconnessione in background.

Postcondizioni:

- Il dispositivo garantisce la continuità operativa in modalità *offline* (fallback) sfruttando la cache locale.
- Il sistema riprende la sincronizzazione automatica non appena la connettività di rete viene ristabilita.

UC 10: Navigazione tra le schermate informative



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 ha dei dati salvati da mostrare.

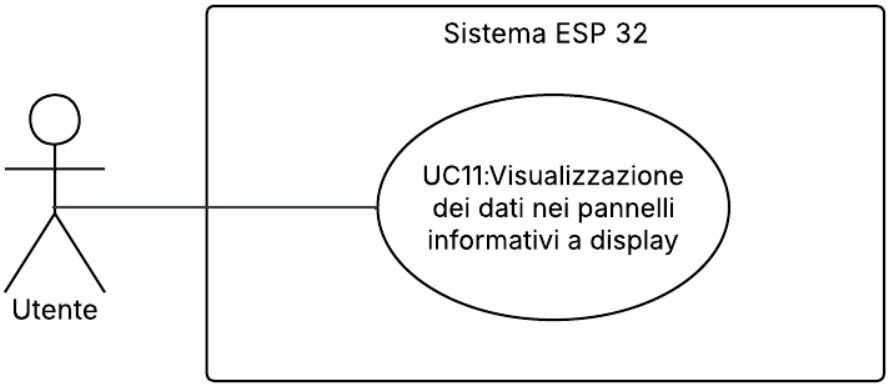
Trigger: Pressione dei pulsanti touch di navigazione o scadenza del timer di rotazione automatica; triplo tocco sullo schermo per abilitare/disabilitare la rotazione.
Descrizione:

1. Il sistema gestisce la rotazione automatica delle pagine in base al timer configurato (default 30 secondi, impostato remoto da CRM RelAi), oppure l'utente interagisce premendo i pulsanti freccia sulla barra di navigazione.
2. Il sistema esegue l'animazione di transizione tra la schermata corrente e la successiva.
3. L'utente visualizza la nuova schermata informativa o la pagina del menu diagnostico.

Postcondizioni:

- L'interfaccia grafica risponde fluidamente ai comandi di navigazione manuale o automatica.
- Lo stato della rotazione (attiva o in pausa) viene aggiornato visivamente sui pulsanti di controllo.

UC 11: Visualizzazione dei dati nei pannelli informativi a display



Attori principali: Utente
Precondizioni:

- Il dispositivo ESP32 è operativo.
- Il dispositivo ESP32 ha dei dati salvati da mostrare.

Trigger: Completamento dell'avvio del dispositivo e ricezione di nuovi dati.
Descrizione:

1. Il sistema esegue il rendering grafico dei pannelli degli eventi strutturati a schede.
2. Ciascun pannello mostra a schermo le informazioni principali dell'evento (nome dell'evento, contatore delle prenotazioni e dei partecipanti).
3. Il sistema genera e mostra la tile laterale associata contenente il codice QR dinamico di riferimento per l'evento.
4. L'utente consulta visivamente le informazioni o interagisce con il touch screen.

Postcondizioni:

- I dati aggiornati dell'evento risultano correttamente renderizzati e visibili a schermo con elevata fluidità visiva.
- Il sistema mantiene la rotazione attiva in base alle impostazioni correnti.

3.3 Requisiti del sistema e stato di completamento

La definizione dei requisiti costituisce il fondamento dell'intero processo di progettazione del sistema. A partire dall'analisi del contesto aziendale, delle esigenze dei clienti e delle criticità tecniche legate all'hardware ESP32, sono stati individuati i requisiti funzionali, non funzionali e di vincolo necessari a garantire la corretta operatività del prototipo. L'obiettivo principale è stato

la realizzazione di una soluzione in grado di mostrare informazioni rilevanti in modo affidabile, semplice e immediatamente leggibile, mantenendo un elevato livello di robustezza anche in condizioni di rete instabile o in presenza di aggiornamenti intermittenti da parte del sistema centrale.

In fase di sviluppo, tutti i requisiti individuati sono stati verificati in modo diretto e progressivo, con riferimento ai casi d'uso precedentemente definiti, alle attività di integrazione con RelAi e ai test eseguiti sul dispositivo. L'esito di questa valutazione ha confermato la completezza del sistema rispetto alle esigenze inizialmente dichiarate. Di conseguenza, il livello di copertura dei requisiti può essere considerato pienamente soddisfatto.

3.3.1 Requisiti funzionali e stato di attuazione

I requisiti funzionali definiscono le capacità che il sistema deve mettere a disposizione dell'organizzazione cliente e dell'utente finale. Essi riguardano in particolare la configurazione del dispositivo, la lettura dei dati di evento, la visualizzazione dei contenuti sul display e la gestione delle operazioni di manutenzione.

Codice	**Requisito**	**Stato**
RF01	Configurazione iniziale della connessione Wi-Fi tramite portale locale e salvataggio persistente delle credenziali.	Completato
RF02	Abbinamento del dispositivo a RelAi e ricezione delle informazioni di configurazione e degli eventi.	Completato
RF03	Visualizzazione sul display dei pannelli informativi relativi a iscritti, partecipanti, prenotazioni e codice QR.	Completato
RF04	Rotazione automatica e navigazione manuale tra le schermate informative.	Completato
RF05	Diagnostica locale del dispositivo con informazioni sullo stato della rete, dell'uptime e del firmware.	Completato
RF06	Operazioni di manutenzione come riavvio, reset e condivisione della rete tramite QR Code.	Completato

Codice	Requisito	Stato
RF07	Gestione di fallback e cache locale in caso di assenza o instabilità della rete.	Completato
RF08	Configurazione remota dei pannelli e della frequenza di aggiornamento da parte del CRM RelAi.	Completato

Tabella 3.1 - Requisiti funzionali e stato di completamento.

3.3.2 Requisiti non funzionali, di qualità e vincolo

I requisiti non funzionali descrivono le caratteristiche di qualità che un sistema embedded deve possedere per essere efficiente, affidabile e facilmente gestibile anche in contesti operativi reali. Tra questi, le criticità più rilevanti riguardano la stabilità del firmware, la reattività dell'interfaccia, l'usabilità del processo di configurazione e la sostenibilità del sistema sul lungo periodo.

Codice	Requisito	Stato
RNF01	Affidabilità del sistema in presenza di disconnessioni temporanee e timeout nelle chiamate verso il CRM.	Completato
RNF02	Robustezza del firmware con gestione degli errori, riconnessione automatica e recupero dalla cache locale.	Completato
RNF03	Prestazioni adeguate per il rendering delle schermate e aggiornamento dei dati in tempo reale.	Completato
RNF04	Usabilità delle operazioni locali per utenti non esperti, con procedure di approccio semplici e trasparenti.	Completato
RNF05	Manutenibilità del codice e separazione dei moduli per facilitare aggiornamenti e test futuri.	Completato
RV01	Esecuzione del sistema su piattaforma ESP32 con vincoli di memoria, potenza computazionale e rendering.	Completato

Codice	Requisito	Stato
RV02	Integrazione con l'ecosistema RelAi e compatibilità con i dati e i protocolli aziendali.	Completato
RV03	Possibilità di configurazione del dispositivo anche senza accesso a internet tramite modalità locale.	Completato

Tabella 3.2 - Requisiti non funzionali e di vincolo.

3.3.3 Conclusione sulla soddisfazione dei requisiti

Dall'analisi condotta durante lo stage emerge che tutti i requisiti inizialmente individuati sono stati soddisfatti in modo completo, sia nei loro aspetti funzionali sia in quelli di qualità e di vincolo. La soluzione ottenuta dimostra quindi la validità dell'approccio progettuale adottato, confermando che il dispositivo embedded può essere integrato efficacemente con il CRM RelAi e utilizzare le risorse hardware disponibili in modo sostenibile ed efficiente.

In particolare, il sistema raggiunge l'obiettivo di trasformare dati aziendali complessi in informazioni facilmente leggibili e accessibili direttamente presso le sedi degli utenti, riducendo l'interazione richiesta e migliorando la fruibilità dell'informazione. La raggiunta completezza dei requisiti rappresenta inoltre una base solida per eventuali sviluppi futuri, come nuove tipologie di pannelli, aggiornamenti OTA o integrazioni con flussi dati ancora più avanzati.

4 Tecnologie utilizzate

Questo capitolo descrive le principali tecnologie hardware e software adottate per realizzare il prototipo di interfaccia informativa integrato con il CRM RelAi. La scelta dei componenti è stata guidata dai requisiti individuati: disponibilità di un display interattivo, possibilità di comunicare tramite rete Wi-Fi, compatibilità con un ambiente di sviluppo per microcontrollori e capacità di aggiornare i dati ricevuti dal sistema centrale.
La soluzione è stata sviluppata in C++ per microcontrollori della famiglia ESP32-S3, utilizzando la libreria grafica LVGL e l'ambiente di sviluppo PlatformIO. Per la comunicazione con il CRM RelAi è stato impiegato un canale WebSocket; la configurazione iniziale della rete avviene invece attraverso un Access Point generato dal dispositivo e un portale di provisioning locale.

4.1 Piattaforma hardware

Il dispositivo principale del prototipo è basato su ESP32-S3 ed è abbinato a un display rotondo da 1,75 pollici. Nel codice, questa configurazione sfrutta le specifiche impostazioni di pinout e driver ottimizzate per il chip, gestendo l'interazione touch e la visualizzazione delle informazioni di stato, contatori e codici QR attraverso la libreria grafica.
In coerenza con il requisito di supporto a più tipologie di display, l'architettura software gestisce tramite PlatformIO e direttive di compilazione condizionale (o ambienti separati) anche una seconda scheda basata su ESP32-S3, abbinata a un display quadrato da 1,8 pollici. Questa separazione consente di mantenere un codice comune per la logica applicativa e le schermate LVGL, differenziando unicamente i driver di inizializzazione dello schermo e delle periferiche specifiche.
Il firmware coordina l'esecuzione dei task di sistema, la connettività Wi-Fi in modalità AP e STA, la ricezione dei messaggi JSON via WebSocket e l'aggiornamento dell'interfaccia. Sulle piattaforme embedded, la gestione della memoria dinamica e la frequenza di refresh dello schermo richiedono particolare attenzione per garantire la fluidità del rendering, specialmente durante il disegno di elementi complessi come i codici QR.



Figura 4.1 - Scheda ESP32-S3 con display rotondo da 1,75 pollici utilizzata come dispositivo principale.



Figura 4.2 - Scheda ESP32-S3 con display quadrato da 1,8 pollici utilizzata per verificare il requisito facoltativo multi-display.

4.2 Linguaggio C++

Il firmware è stato implementato in C++, sfruttando i paradigmi della pro-

grammazione orientata agli oggetti per suddividere l'applicazione in moduli riutilizzabili e manutenibili. Questa scelta architetturale facilita l'intergazione tra i driver delle periferiche, lo stack di rete e la libreria grafica, isolando le diverse responsabilità del sistema.
La gestione delle risorse a memoria limitata dell'ESP32-S3 ha richiesto un'attenta ottimizzazione nella scrittura del codice, volto a garantire un bilanciamento efficiente tra i task di elaborazione dati, la reattività la reattività della rete e la fluidità del rendering grafico sull'interfaccia.



Figura 4.3 - Logo del linguaggio di programmazione C++.

4.3 PlatformIO

Per configurare e gestire il progetto è stato utilizzato PlatformIO, ambiente di sviluppo integrato con Visual Studio Code. Lo strumento permette di dichiarare la piattaforma hardware di destinazione e le dipendenze del firmware, oltre a gestire le operazioni di compilazione e caricamento del programma sulla scheda.
La configurazione del progetto raccoglie i parametri necessari a riprodurre la toolchain e distingue le impostazioni comuni del firmware da quelle legate alle singole schede. PlatformIO è stato inoltre utilizzato nel ciclo iterativo di sviluppo: dopo ogni modifica, il firmware viene compilato e caricato sul dispositivo per verificare il comportamento effettivo dell'interfaccia e delle funzioni di rete.



Figura 4.4 - Logo dell'ambiente di sviluppo PlatformIO.

4.4 LVGL e interfaccia grafica

Per la realizzazione dell'interfaccia grafica è stata adottata LVGL (*Light and Versatile Graphics Library*), una libreria ottimizzata per sistemi embedded con display. LVGL mette a disposizione widget predefiniti e strumenti per la costruzione di schermate interattive, la gestione degli eventi di input e il rendering efficiente delle aree dello schermo, evitando di implementare da zero le primitive grafiche di basso livello.
Nel progetto, la libreria viene impiegata per strutturare i pannelli informativi, visualizzare metriche, contatori e codici QR, gestire i menu di navigazione e le schermate diagnostiche. L'adozione di un pattern architetturale pulito separa la logica applicativa dai componenti di presentazione, facilitando l'aggiornamento dinamico dei contenuti in base ai messaggi ricevuti dal CRM e garantendo un'interazione coerente su entrambe le tipologie di display.
L'adattamento dell'interfaccia alle differenti geometrie dello schermo ha richiesto una configurazione mirata dei driver di visualizzazione e del modulo touch. Poiché il layout deve risultare perfettamente leggibile sia sul pannello circolare che su quello quadrato, gli elementi grafici sono stati disposti tenendo conto dei vincoli fisici dello spazio utile, valorizzando le peculiarità di ciascun formato senza compromettere l'usabilità.



Figura 4.5 - Esempio dell'interfaccia realizzata con LVGL sul display del dispositivo.

4.5 Connettività e protocolli

La connettività wireless si basa sul protocollo Wi-Fi e prevede una gestione progressiva degli stati di connessione. All'avvio, il dispositivo si trova inizialmente in uno stato non connesso e verifica la presenza di credenziali memorizzate nella memoria non volatile tramite la libreria Preferences, avviando la connessione in modalità station (STA) in background.
Nel caso in cui le credenziali non siano configurate o il tentativo di connessione iniziale fallisca, il sistema attiva la modalità di provisioning Access Point (AP), generando una rete dedicata (ESP32-Config) e avviando un server HTTP locale supportato da un DNSServer per la gestione del captive portal. Questo permette all'utente di accedere al portale e visualizzare la scansione delle reti circostanti per configurare i parametri corretti.
Una volta stabilita con successo la connessione alla rete dell'organizzazione, il dispositivo adotta un'architettura ibrida in modalità WIFI_AP_STA. In questa configurazione, l'Access Point rimane permanentemente attivo e accessibile in parallelo alla modalità STA, offrendo all'utente la costante possibilità di verificare, modificare o reinserire le credenziali in ogni momento senza interrompere il funzionamento ordinario.
La comunicazione in tempo reale con il CRM RelAi sfrutta un canale WebSocket persistente. A differenza delle richieste HTTP tradizionali, la connessione bidirezionale consente al dispositivo di ricevere istantaneamente aggiornamenti e comandi remoti, quali la sincronizzazione dei dati, il riavvio o il reset. I messaggi scambiati sono strutturati in formato JSON e vengono

elaborati dal firmware per aggiornare dinamicamente lo stato del sistema.
In caso di interruzione temporanea della connettività o di irraggiungibilità del CRM, il sistema si avvale dei dati validi conservati nella cache locale, avviando al contempo routine di riconnessione automatica. Questo meccanismo di fallback garantisce la continuità visiva delle informazioni sui pannelli e riduce l'impatto dei disservizi di rete sulla fruibilità del prototipo.

4.6 Persistenza e gestione dei dati

Le impostazioni necessarie al funzionamento del dispositivo e i dati utilizzati per il recupero dell'ultimo stato valido sono conservati in memoria non volatile. Nei casi d'uso la memoria persistente è identificata come NVS (*Non-Volatile Storage*): essa permette di mantenere le configurazioni anche in seguito a un riavvio. La procedura di reset elimina le informazioni previste e riporta il dispositivo alla fase di configurazione iniziale.
I dati ricevuti dal CRM vengono elaborati dal firmware e confrontati con quelli già disponibili. Quando è necessario, la copia locale viene aggiornata e l'interfaccia viene ridisegnata per riflettere i nuovi contenuti. La cache locale svolge inoltre la funzione di fallback: in assenza di connettività permette di continuare a visualizzare le informazioni dell'ultima sincronizzazione valida.

4.7 Controllo di versione e documentazione

Lo sviluppo e la documentazione richiedono strumenti che consentano di mantenere tracciabili le modifiche, organizzare i materiali del progetto e produrre elaborati aggiornabili. A questo scopo sono stati utilizzati Gitea per il controllo di versione e Typst per la stesura della documentazione.

4.7.1 Gitea

Gitea è stato utilizzato come piattaforma di gestione dei repository del progetto firmware. Il controllo di versione permette di conservare la cronologia delle modifiche, isolare gli interventi in branch dedicati e sottoporre le modifiche a revisione tramite pull request e code review. Come descritto nel capitolo dedicato ai processi, questa organizzazione supporta la collaborazione con il tutor e facilita l'individuazione delle modifiche introdotte durante le diverse fasi di sviluppo.

4.7.2 Typst per la redazione della documentazione

La documentazione tecnica è stata redatta in Typst, un sistema di composizione tipografica basato su markup. L'elaborato è suddiviso in file sorgente

distinti per i capitoli e utilizza un file principale per configurare il documento e includere i contenuti. Questa struttura modulare rende più agevole aggiornare le singole parti e mantenere uniformi titoli, riferimenti, figure e tabelle.
La compilazione dei sorgenti produce il documento PDF destinato alla consultazione e alla consegna. L'uso di un formato testuale versionabile permette inoltre di tracciare le revisioni della documentazione insieme ai materiali del progetto e di rigenerare il PDF dopo gli aggiornamenti.

4.8 Riepilogo dello stack tecnologico

La tabella seguente riassume i principali elementi tecnologici impiegati e il ruolo svolto all'interno del prototipo.

Ambito	**Tecnologia**	**Utilizzo nel progetto**
Linguaggio	C++	Implementazione del firmware orientata agli oggetti e suddivisa in moduli manutenibili.
Microcontrollore	Famiglia ESP32-S3	Esecuzione del firmware e gestione delle periferiche su schede dotate rispettivamente di display rotondo da 1,75 pollici e quadrato da 1,8 pollici.
Interfaccia grafica	LVGL	Composizione delle schermate, gestione dei componenti visuali e degli input touch.
Toolchain	PlatformIO	Configurazione del progetto, compilazione, caricamento e verifica del firmware.
Rete locale	Wi-Fi AP e STA	Gestione ibrida permanente con Access Point locale per il provisioning e connessione station verso la rete dell'organizzazione.
Comunicazione cloud	WebSocket e JSON	Ricezione bidirezionale in tempo reale di aggiornamenti e comandi remoti dal CRM RelAi.
Persistenza	NVS e LittleFS	Conservazione delle configurazioni tramite Preferences, cache

Ambito	Tecnologia	Utilizzo nel progetto
		eventi e salvataggio del logo su file system.
Controllo versione	Gitea	Gestione dei repository, cronologia delle modifiche, branch e revisioni del codice.
Documentazione	Typst	Redazione modulare della documentazione tecnica e generazione del documento PDF.

Tabella 4.1 - Riepilogo delle tecnologie utilizzate.

Le tecnologie descritte costituiscono la base per le successive attività di progettazione e implementazione del firmware. In particolare, la piattaforma hardware basata su ESP32-S3 e LVGL definiscono il contesto della presentazione grafica, mentre l'architettura di rete ibrida Wi-Fi, i canali WebSocket e i meccanismi di persistenza locale consentono un'integrazione resiliente con il CRM RelAi e la continuità del servizio.